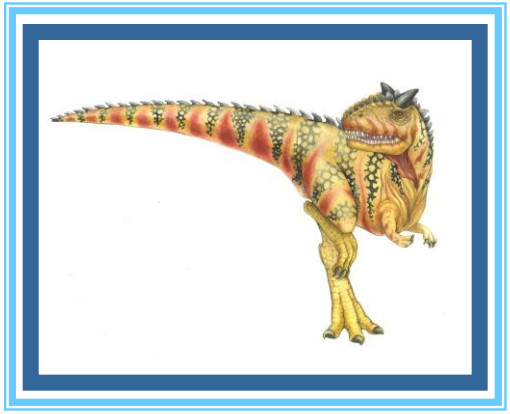
Chapter 8: Main Memory

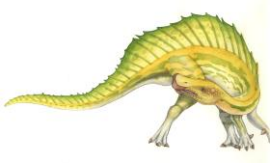


Silberschatz, Galvin and Gagne ©2018



Chapter 8: Memory Management

- Background
- Contiguous Memory Allocation
- Swapping
- Fragmentation



Silberschatz, Galvin and Gagne ©2018



Objectives

- To provide a detailed description of various ways of organizing memory hardware
- To discuss various memory-management techniques,
- To provide a detailed description of the Intel Pentium, which supports both pure segmentation and segmentation with paging



Silberschatz, Galvin and Gagne ©2018



Background

- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of:
  - addresses + read requests, or
  - address + data and write requests
- Register access is done in one CPU clock (or less)
- Main memory can take many cycles.
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation

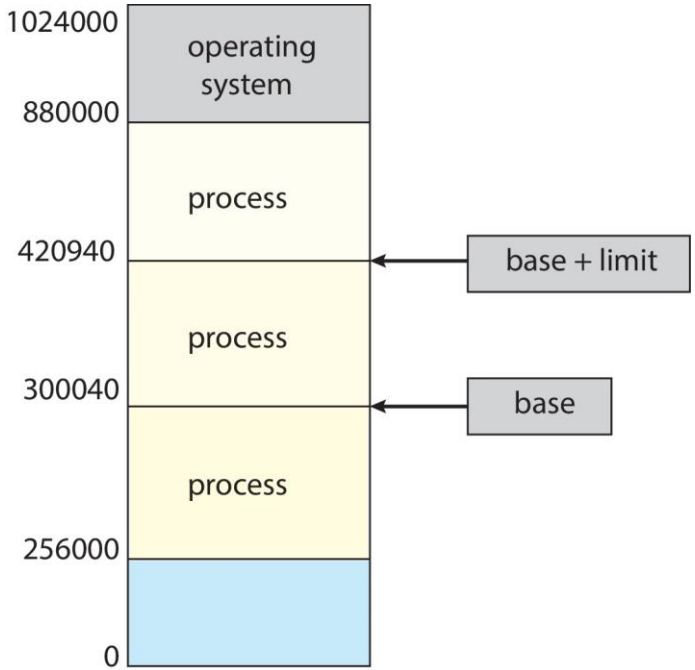


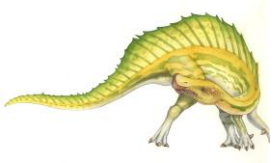
Silberschatz, Galvin and Gagne ©2018



Protection

- Need to censure that a process can access only access those addresses in its address space.
- We can provide this protection by using a pair of **base** and **limit registers** define the logical address space of a process



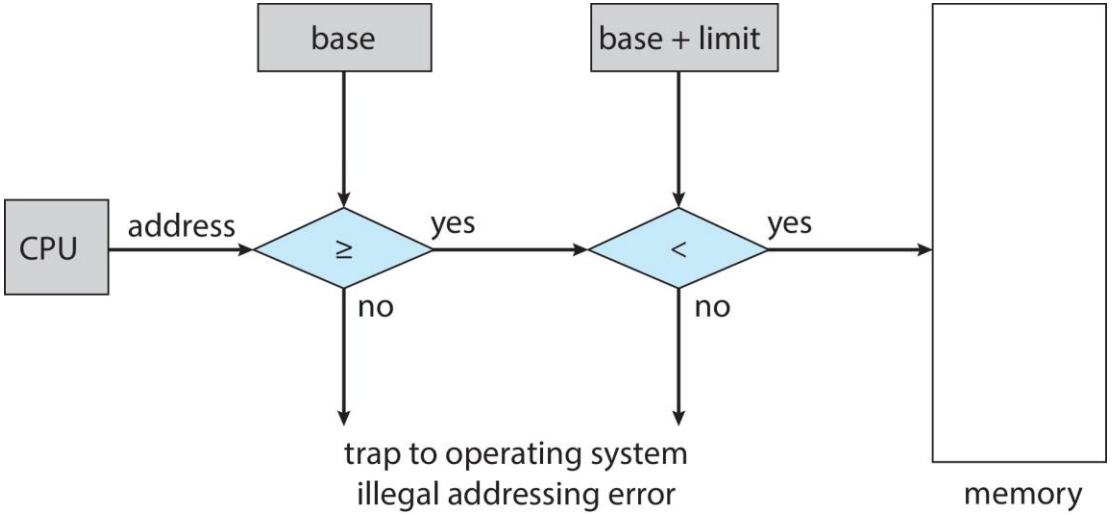


Silberschatz, Galvin and Gagne ©2018



Hardware Address Protection

- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



- the instructions to loading the base and limit registers are privileged



Silberschatz, Galvin and Gagne ©2018



Logical vs. Physical Address Space

- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
  - **Logical address** – generated by the CPU; also referred to as **virtual address**
  - **Physical address** – address seen by the memory unit
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program

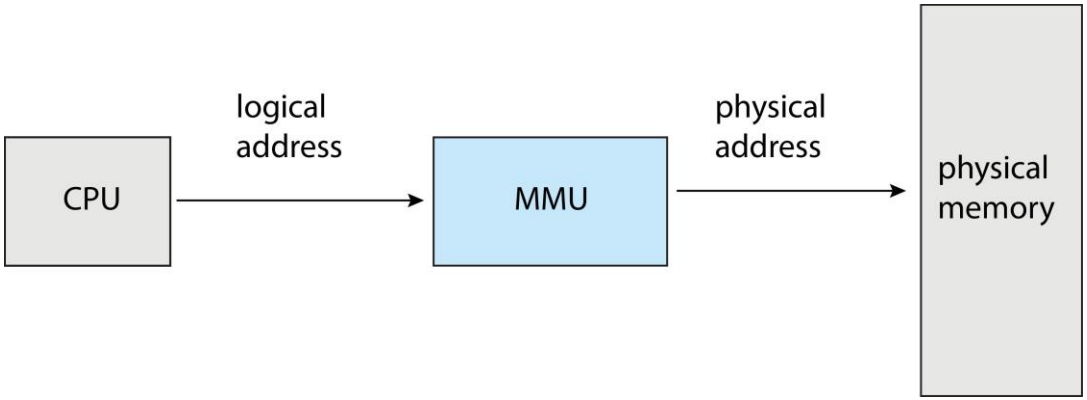


Silberschatz, Galvin and Gagne ©2018



Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address



- Many methods possible, covered in the rest of this chapter

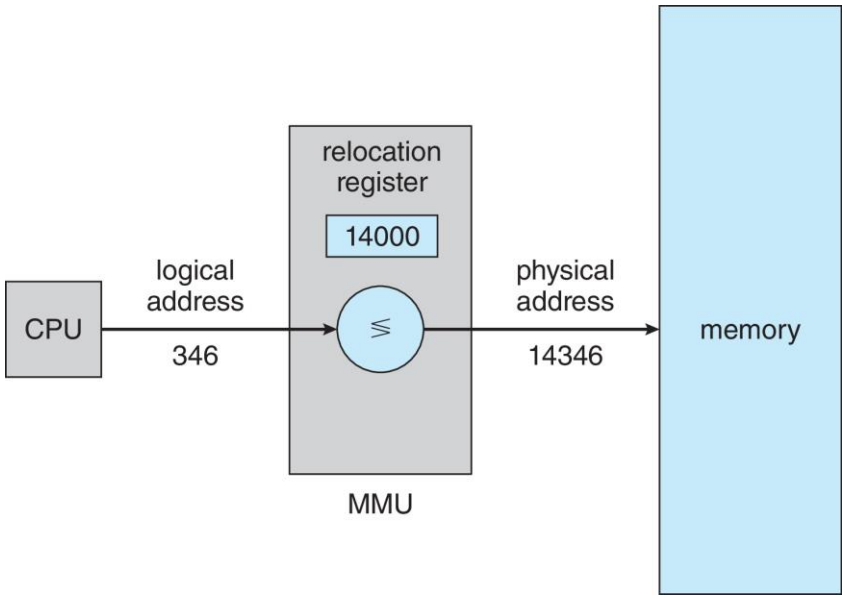


Silberschatz, Galvin and Gagne ©2018



Memory-Management Unit (Cont.)

- Consider simple scheme. which is a generalization of the base-register scheme.
- The base register now called **relocation register.**
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory





Silberschatz, Galvin and Gagne ©2018



Swapping

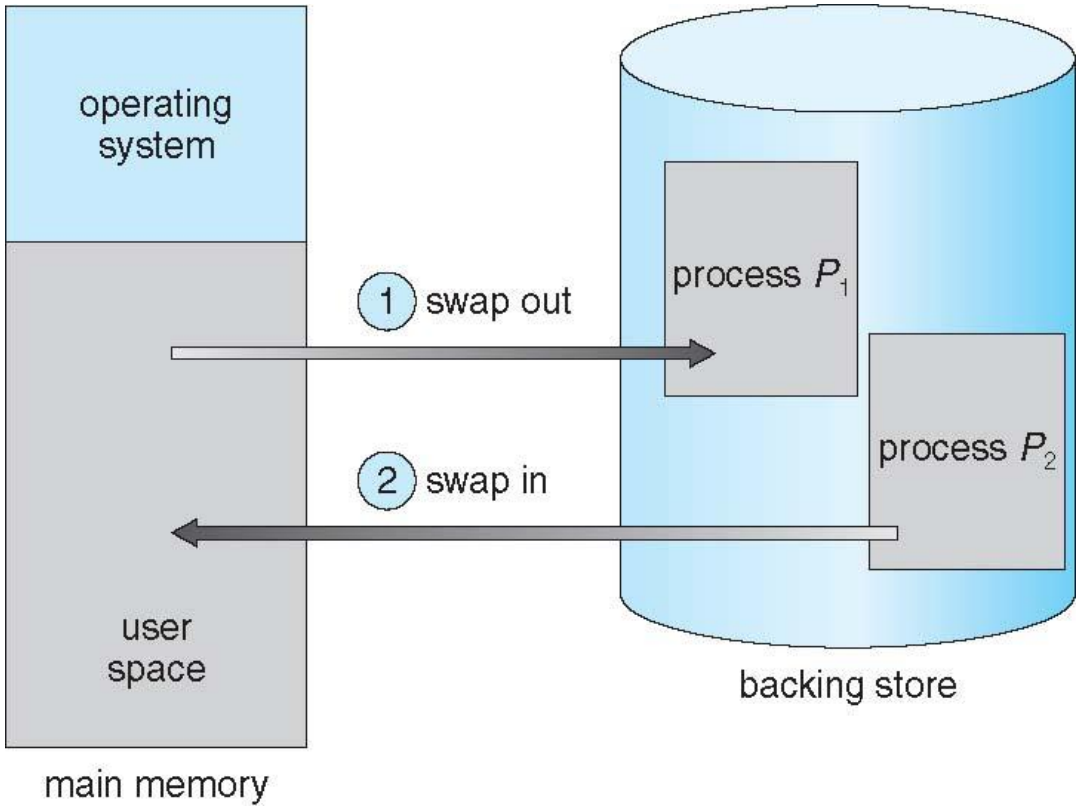
- A process can be **swapped** temporarily out of memory to a backing store, and then brought **back** into memory for continued execution
  - Total physical memory space of processes can exceed physical memory
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk

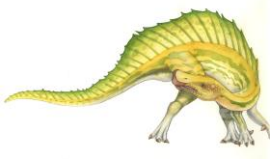


Silberschatz, Galvin and Gagne ©2018



Schematic View of Swapping





Silberschatz, Galvin and Gagne ©2018



Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used
- First fit analysis reveals that given *N* blocks allocated, 0.5 *N* blocks lost to fragmentation
  - 1/3 may be unusable -> **50-percent rule**



Silberschatz, Galvin and Gagne ©2018



Fragmentation (Cont.)

- Reduce external fragmentation by **compaction**
  - Shuffle memory contents to place all free memory together in one large block
  - Compaction is possible *only* if relocation is dynamic, and is done at execution time
  - I/O problem
    - Latch job in memory while it is involved in I/O
    - Do I/O only into OS buffers
- Now consider that backing store has same fragmentation problems



Silberschatz, Galvin and Gagne ©2018